

FTP Server - RFC 959 Summary (Beginner Friendly)

This document summarizes the core points of [RFC 959](#), which defines the FTP (File Transfer Protocol). This will help guide the development of our custom FTP server in Python.

What is FTP?

FTP stands for **File Transfer Protocol**. It is a way for two computers to transfer files over the Internet or a network. One acts as a **server**, and the other as a **client**.

 *Real-World Analogy:* Think of it like a courier service. The FTP client sends a request ("I want this file") and the FTP server delivers it.

Key Concepts in RFC 959

1. Control vs Data Connection

- **Control Connection:** Always open. Used to send commands and responses.
- **Data Connection:** Opened only when data (like files or directory listing) is transferred.

Example: You call a shop (control connection) to say what you want. The shop sends a delivery guy (data connection) to deliver your package.

2. Authentication

- **USER** : Send the username.
- **PASS** : Send the password.

 RFC 959 supports **plain-text** credentials. No encryption.

3. Directory Commands

- **CWD** : Change working directory
 - **PWD** : Print current directory
 - **MKD** : Make a new directory
 - **RMD** : Remove directory
 - **LIST** : List files in directory
-

4. File Commands

- `RETR filename` : Retrieve (download) file from server
 - `STOR filename` : Store (upload) file to server
 - `DELE filename` : Delete a file
-

5. Active vs Passive Mode

- **Active Mode (PORT)**: Client tells server where to connect for data.
- **Passive Mode (PASV)**: Server tells client where to connect. Useful behind firewalls.

Example: In Active mode, you say "come to my house". In Passive mode, the server says "come to my shop".

6. FTP Replies (Status Codes)

- `200` : OK
- `220` : Service ready
- `221` : Goodbye
- `230` : Logged in
- `331` : Username OK, need password
- `530` : Not logged in (auth failed)
- `550` : File not found or no permission

These codes help the client know what's going on.

7. Transfer Types

- `TYPE A` : ASCII text
- `TYPE I` : Image (binary)

Use `TYPE I` for media and executables.

8. Abort or Stop

- `ABOR` : Abort transfer (like canceling a file in progress)
-

9. Structure of Commands

Commands are **text-based**, sent over TCP port **21** (control connection). Responses are also text + code.

Example:

```
Client → USER john
Server → 331 Username OK, need password
Client → PASS 12345
Server → 230 User logged in
```

Why RFC 959 is Enough for Our Project

- It covers everything to make a working FTP server
- No encryption or GUI is needed for now
- Focuses on core logic: commands, sockets, file handling

Final Thoughts

This is the foundation of your FTP server. You'll build a server that:

- Accepts connections
- Handles user login
- Supports basic FTP commands
- Transfers files using sockets

Stay tuned for code implementation after this documentation phase!

Maintained by: Ashutosh Sonawane – Embedded Developer (FTP Server in Python Project)